Logic Engine Design — Comprehensive Implementation Guide

From Lattice Foundations to Relational Surface Syntax

Prologos Project

2026-02-24

Contents

1 Executive Summary **5**
1.1 Infrastructure Gap Analysis 6
1.2 Critical Path 6

2 Phase 1: Lattice Trait + Standard Instances **7**
2.1 1.1 Goal 7
2.2 1.2 The `Lattice` Trait 7
2.3 1.3 Standard Lattice Instances 7
2.3.1 1.3.1 `FlatLattice A` — Three-Point Lattice 7
2.3.2 1.3.2 `SetLattice A` — Powerset Lattice (Set Union) . 8
2.3.3 1.3.3 `MapLattice K V` — Pointwise Map Lattice . . . 8
2.3.4 1.3.4 `IntervalLattice` — Numeric Intervals 9
2.3.5 1.3.5 `BoolLattice` — Two-Point Lattice 9
2.4 1.4 Racket-Level Implementation 9
2.5 1.5 `champ-insert-join` — Racket-Level Helper 10
2.6 1.6 New Files 10
2.7 1.7 Tests (~25) 10
2.8 1.8 Dependencies 11

3 Phase 2: Persistent Propagator Network **11**
3.1 2.1 Goal 11
3.2 2.2 Architecture 12
3.3 2.3 Core Data Structures (All Persistent) 12
3.3.1 2.3.1 Identity Types 12
3.3.2 2.3.2 `prop-cell` — Propagator Cell (Immutable) . . . 13

3.3.3 2.3.3 `propagator` — Monotone Function (Immutable) 13
3.3.4 2.3.4 `prop-network` — The Network as Value 13
3.4 2.4 Pure Operations 14
3.5 2.5 Concrete `fire-fn` Example: Adder Propagator 16
3.6 2.6 `run-to-quiescence` — Pure Loop 17
3.7 2.7 Contradiction Handling (Per-Cell `contradicts?` Predicate) 18
3.8 2.8 LVars Are Subsumed by Cells 19
3.9 2.9 AST Nodes (~12) 20
3.10 2.10 New/Modified Files 20
3.11 2.11 Tests (~60) 20
3.12 2.12 Dependencies 21

4 Phase 3: PropNetwork as Prologos Type 22
4.1 3.1 Goal 22
4.2 3.2 Why a Separate Phase 22
4.3 3.3 Type Signatures 22
4.4 3.4 LVar Operations as Library Functions 23
4.5 3.5 AST Nodes (12) 23
4.6 3.6 New/Modified Files 23
4.7 3.7 Tests (~50) 24
4.8 3.8 Dependencies 24

5 Phase 4: UnionFind — Persistent Disjoint Sets 24
5.1 4.1 Goal 24
5.2 4.2 Design 25
5.3 4.3 Key Properties 26
5.4 4.4 Integration with Logic Engine: UF vs Cell Division of Labor 26
5.5 4.5 AST Nodes (~6) 27
5.6 4.6 New/Modified Files 27
5.7 4.7 Tests (~30) 27
5.8 4.8 Dependencies 28

6 Phase 5: Persistent ATMS Layer — Hypothetical Reasoning 28
6.1 5.1 Goal 28
6.2 5.2 Core Data Structures (All Persistent) 28
6.2.1 `assumption` — Hypothetical Premise 28
6.2.2 `supported-value` — Value + Justification 28
6.2.3 `tms-cell` — Truth-Maintained Cell (Immutable) . . . 29
6.2.4 `atms` — The Persistent ATMS 29
6.3 5.3 Pure Operations 29

6.4 5.4 The `amb` Operator (Pure) . . . 30
6.5 5.5 Two-Tier Mode: Lazy ATMS Activation . . . 31
6.6 5.6 Contradiction Handler (Dependency-Directed Backtracking) 32
6.7 5.7 Answer Collection (Pure) . . . 32
6.8 5.8 AST Nodes (~10) . . . 33
6.9 5.9 New/Modified Files . . . 33
6.10 5.10 Tests (~50) . . . 33
6.11 5.11 Dependencies . . . 34

7 Phase 6: Tabling — SLG-Style Memoization 34
7.1 6.1 Goal . . . 34
7.2 6.2 Design (XSB-Style SLG Resolution) . . . 34
7.3 6.3 Table Lifecycle . . . 35
7.4 6.4 Core Data Structures (Persistent) . . . 35
7.5 6.5 Lattice Answer Modes . . . 36
7.6 6.6 Spec Metadata Integration . . . 36
7.7 6.7 AST Nodes (~8) . . . 36
7.8 6.8 New/Modified Files . . . 37
7.9 6.9 Tests (~40) . . . 37
7.10 6.10 Dependencies . . . 37

8 Phase 7: Surface Syntax — `defr`, `rel`, `solve`, `&>` 38
8.1 7.1 Goal . . . 38
8.2 7.2 Reader Changes . . . 38
8.3 7.3 Parser Changes . . . 38
8.4 7.4 Elaboration . . . 38
8.4.1 7.4.1 `defr` Elaboration . . . 38
8.4.2 7.4.2 `solve` Elaboration (Functional-Relational Bridge) 39
8.5 7.5 Grammar Updates . . . 40
8.6 7.6 Compile-Time Stratification Check . . . 41
8.7 7.7 AST Nodes (~15) . . . 42
8.8 7.8 New/Modified Files . . . 42
8.9 7.9 Tests (~80) . . . 43
8.10 7.10 Dependencies . . . 44

9 Phase Summary 44

10 Interaction with Existing Infrastructure 45
10.1 Metavar System . . . 45
10.2 Trait System . . . 45

10.3 Spec Metadata . . . 46
10.4 Warnings . . . 46
10.5 QTT / Multiplicities . . . 46
10.6 Collections . . . 46

11 Appendix A: Resolution by Example — `ancestor` as Propagators 47
11.1 A.1 Source Program . . . 47
11.2 A.2 Step 1: Table Creation . . . 47
11.3 A.3 Step 2: Clause 1 → Producer Propagator . . . 47
11.4 A.4 Step 3: Clause 2 → Producer Propagator . . . 48
11.5 A.5 Step 4: Run to Quiescence . . . 49
11.6 A.6 Step 5: Answer Extraction . . . 49

12 Appendix B: End-to-End Query Walkthrough (All Three Layers) 49
12.1 B.1 Source Program . . . 49
12.2 B.2 Compile-Time: Stratification Check . . . 50
12.3 B.3 Runtime: Layer 1 — PropNetwork (Stratum 0) . . . 50
12.4 B.4 Runtime: Layer 3 — Stratum Boundary . . . 51
12.5 B.5 Runtime: Layer 2 — ATMS (if needed) . . . 51
12.6 B.6 Summary: Which Layer Handles What . . . 52

13 Performance Expectations 52
13.1 Cost Model . . . 52
13.2 Benchmark Targets (to validate during implementation) . . . 53
13.3 When Performance Matters Most . . . 53

14 What This Design Does NOT Cover 53
14.1 Elaborator Refactoring (Phase 1 of Research Doc) . . . 53
14.2 Galois Connections / Domain Embeddings (Phase 6 of Research Doc) . . . 54
14.3 Full Stratified Evaluation Runtime (Phase 4 of Research Doc) 54
14.4 CRDTs / Distributed Logic . . . 54
14.5 QuickCheck / Property Testing . . . 54

15 Architectural Decision: Persistent Networks 54
15.1 The Problem with Mutable Propagator Networks . . . 54
15.2 The Persistent Solution . . . 55
15.3 LVar Elimination . . . 55

16 Key Lessons from Prior Work **56**

17 References **56**
17.1 Phase 1 (Lattice) 56
17.2 Phase 2 (Propagators) 57
17.3 Phase 3 (LVars) 57
17.4 Phase 4 (UnionFind) 57
17.5 Phase 5 (ATMS) 57
17.6 Phase 6 (Tabling) 57
17.7 Phase 7 (Surface) 57

1 Executive Summary

This document is the implementation guide for Prologos's logic engine — the substrate on which the relational language (`defr`, `rel`, `solve`) will run. It follows the research-design-implementation methodology that produced the Extended Spec Design (`2026-02-22_EXTENDED_SPEC_DESIGN.org`).

The design is grounded in two research documents:

- Towards a General Logic Engine on Propagators — three-layer architecture
- Implementation Guide: Core Data Structures — Section 8 (lattice integration)

And the relational language vision:

- Relational Language Vision — surface syntax decisions

The engine is built bottom-up in 7 phases:

1. **Lattice Trait + champ-insert-join** — the algebraic foundation
2. **Persistent PropNetwork** (Racket-level) — the monotonic data plane as value
3. **PropNetwork as Prologos Type** — expose network ops to the type system
4. **UnionFind** — persistent disjoint sets for unification
5. **Persistent ATMS** — hypothetical reasoning as value
6. **Tabling** — SLG-style memoization for completeness

7. **Surface Syntax** — `defr`, `rel`, `solve`, `&>`

Key architectural decision: the entire propagator network and ATMS are **persistent/immutable values** backed by CHAMP maps. Backtracking = keep old reference (O(1)). Snapshots = free. Network mobility = serialize value. LVars are subsumed by PropNetwork cells (join-on-write semantics).

Phases 1-4 are infrastructure with no surface syntax changes. Phase 5-6 add runtime logic capabilities. Phase 7 adds the user-facing language.

1.1 Infrastructure Gap Analysis

Component	Status	What Exists	What's N
Persistent collections	COMPLETE	PVec, Map, Set, List, LSeq	—
CHAMP lattice helper	COMPLETE	`champ-insert-join` in champ.rkt	—
Transient builders	COMPLETE	TVec, TMap, TSet, with-transient	—
Trait system	COMPLETE	Registry, resolution, bundles	Lattice tr
Property system	Phase 1 COMPLETE	Storage, flattening, accessors	QuickChe
Spec metadata	Phase 1 COMPLETE	:examples, :deprecated, :doc	:tabled, :a
Metavar system	COMPLETE	4 parallel stores, constraints	Refactor t
Lattice trait	NOT STARTED	—	trait + 5-
Persistent PropNetwork	NOT STARTED	—	Immutabl
PropNetwork Prologos type	NOT STARTED	—	AST node
UnionFind	NOT STARTED	—	Persistent
Persistent ATMS	NOT STARTED	—	Immutabl
Tabling	NOT STARTED	—	Producer/
Stratification	NOT STARTED	—	SCC deco
Logic syntax	NOT STARTED	—	defr, rel, s

1.2 Critical Path

```
Phase 1 (Lattice + champ-insert-join)
  ↓
Phase 2 (Persistent PropNetwork, Racket-level) ← depends on Phase 1
  ↓
Phase 3 (PropNetwork as Prologos Type) ← depends on Phase 2
  ↓                    ↓
Phase 4 (UnionFind) ← can proceed in parallel with Phase 3
  ↓                    ↓
Phase 5 (Persistent ATMS) ← depends on Phase 3
  ↓
```

```
Phase 6 (Tabling) ← depends on Phase 3, Phase 5
  ↓
Phase 7 (Surface Syntax) ← depends on ALL previous phases
```

2 Phase 1: Lattice Trait + Standard Instances

2.1 1.1 Goal

Establish the `Lattice` trait — the algebraic foundation for monotonic computation. Every propagator cell, LVar, and ATMS label set requires a lattice domain. By defining `Lattice` as a standard Prologos trait, we get automatic dictionary resolution at both compile-time and runtime.

2.2 1.2 The `Lattice` Trait

```
;; in lib/prologos/core/lattice.prologos
(ns prologos::core::lattice :no-prelude)

trait Lattice {A : Type}
  bot  : A                      ;; bottom element (no information)
  join : A -> A -> A            ;; least upper bound (merge information)
  leq  : A -> A -> Bool         ;; lattice ordering

;; Laws (to be checked via property testing in Phase 2+):
;;   join is commutative:  join a b = join b a
;;   join is associative:  join (join a b) c = join a (join b c)
;;   join is idempotent:   join a a = a
;;   bot is identity:      join bot a = a
;;   leq is consistent:    leq a b = (join a b == b)
```

2.3 1.3 Standard Lattice Instances

2.3.1 1.3.1 `FlatLattice A` — Three-Point Lattice

The simplest useful lattice: bottom → exactly one value → top (contradiction). Used for basic unification cells where a variable is either unknown, bound to exactly one value, or in conflict.

```
;; in lib/prologos/core/lattice-instances.prologos

;; FlatLattice wraps any Eq type into a flat lattice
```

```
;;  → value →
functor FlatLattice {A : Type}
  :where (Eq A)
  :unfolds <(Option [Either A Unit])>
  ;; none = , some (left v) = value, some (right unit) =

instance Lattice [FlatLattice A] where (Eq A)
  bot  = none
  join = flat-join
  leq  = flat-leq

defn flat-join [a b]
  match [a b]
    | [none _]                      -> b
    | [_ none]                      -> a
    | [[some [left v1]] [some [left v2]]]
      -> if [eq? v1 v2] a [some [right unit]]
    | _                             -> [some [right unit]]
```

2.3.2 1.3.2 `SetLattice A` — Powerset Lattice (Set Union)

Growing sets: elements can only be added, never removed. Join = union. Used for Herbrand interpretations (sets of ground facts) and LVar-Sets.

```
;; SetLattice: growing sets with union as join
functor SetLattice {A : Type}
  :unfolds [Set A]

instance Lattice [SetLattice A]
  bot  = #{}
  join = set-union
  leq  = set-subset?
```

2.3.3 1.3.3 `MapLattice K V` — Pointwise Map Lattice

Maps where each key has an independent lattice-valued value. Join = pointwise join of values. Used for LVar-Maps and substitution stores.

```
;; MapLattice: pointwise lattice on maps
functor MapLattice {K V : Type}
  :where (Lattice V)
```

```
  :unfolds [Map K V]

instance Lattice [MapLattice K V] where (Lattice V)
  bot  = {}
  join = map-lattice-join
  leq  = map-lattice-leq

defn map-lattice-join [m1 m2]
  ;; For keys in both: join values. For keys in one: include.
  [merge-with join m1 m2]
```

2.3.4 1.3.4 `IntervalLattice` — Numeric Intervals

Intervals `[lo, hi]` with intersection as meet, hull as join. Used for numeric constraint propagation (CLP(R/FD)).

```
;; IntervalLattice for numeric constraints
deftype Interval
  mk-interval : Rat -> Rat -> Interval    ;; lo, hi (lo <= hi)
  ;; Special: bot = (-, +) = unconstrained
  ;; Special: top = empty = contradicted

instance Lattice Interval
  bot  = interval-unconstrained
  join = interval-intersect    ;; narrowing: more constrained = higher
  leq  = interval-subsumes?   ;; [a,b]  [c,d] iff [c,d]  [a,b]
```

2.3.5 1.3.5 `BoolLattice` — Two-Point Lattice

The simplest non-trivial lattice: false $<$ true. Join $=$ OR. Used as building block and for boolean constraint propagation.

```
instance Lattice Bool
  bot  = false
  join = bool-or
  leq  = bool-implies?
```

2.4 1.4 Racket-Level Implementation

At the Racket level, lattice operations are dispatched via the existing trait system. The `Lattice` trait follows the standard pattern:

Component	How
Trait struct	`trait-meta` in `current-trait-registry`
Instances	`(impl-entry 'Lattice ...)` in `current-impl-registry`
Dictionary type	Sigma type: `(bot . (join . leq))` (3 methods)
Resolution	Standard `resolve-trait-constraints!` pipeline

No new AST nodes needed for Phase 1. The trait and instance declarations use existing infrastructure.

2.5 1.5 `champ-insert-join` — Racket-Level Helper

A lattice-aware CHAMP insert that joins on collision, rather than replacing. This is the foundational Racket-level helper that enables the PropNetwork's cells map to do lattice merge natively.

```
;; In racket/prologos/champ.rkt (already implemented)

;; champ-insert-join : champ-root hash key val (val val → val) → champ-root
(define (champ-insert-join root hash key val join-fn)
  (let ([existing (champ-lookup root hash key)])
    (if (eq? existing 'none)
        (champ-insert root hash key val)
        (champ-insert root hash key (join-fn existing val)))))
```

This enables the propagator network to store cell values in a CHAMP map with join-on-write semantics: writing to a cell computes `merge-fn(old-value, new-value)` and inserts the result.

2.6 1.6 New Files

File	Purpose
`lib/prologos/core/lattice.prologos`	`Lattice` trait declaration
`lib/prologos/core/lattice-instances.prologos`	Standard instances
`tests/test-lattice.rkt`	Trait resolution + law tests

2.7 1.7 Tests (~25)

- Lattice trait registered correctly
- FlatLattice: bot is identity, join of same values, join of different → top

- SetLattice: bot = empty, join = union, leq = subset
- MapLattice: pointwise join, partial maps merge correctly
- BoolLattice: basic operations
- Trait resolution: `(Lattice [FlatLattice Nat])` resolves correctly
- Laws: commutativity, associativity, idempotency of join (6 tests per instance)

2.8 1.8 Dependencies

- Existing trait system (`process-trait`, `process-impl`)
- Existing collection types (`Set`, `Map` for SetLattice, MapLattice)
- `Eq` trait (for `FlatLattice`)
- `champ-insert-join` in `champ.rkt` (already implemented)
- **No new AST nodes**
- **No changes to typing-core.rkt**

—

3 Phase 2: Persistent Propagator Network

3.1 2.1 Goal

Implement the monotonic data plane as a **persistent, immutable value**. The entire propagator network — cells, propagators, worklist, and metadata — is a single Racket struct backed by CHAMP maps. All operations are pure functions: they take a network and return a new network.

This is a critical design choice: the propagator network is a **first-class value** that can be snapshotted (free), backtracked (O(1) — keep old reference), migrated (serialize and send), and compared (structural equality).

3.2 2.2 Architecture

```
prop-network (persistent value)

cells : champ-root (cell-id → cell)
propagators : champ-root (prop-id)
worklist : list of prop-id
merge-fns : champ-root (cell-id → fn)
fuel : Nat
contradiction : #f | cell-id
next-cell-id : Nat
next-prop-id : Nat

All operations return NEW networks:
net-new-cell   : Net → ... → (Net,Id)
net-cell-write : Net → Id → V → Net
run-to-quiescence : Net → Net
```

Backtracking model:

```
let net0 = (make-prop-network)
let net1 = (net-cell-write net0 cell-a 42)
let net2 = (net-cell-write net1 cell-b 99)

;; Backtrack to net1: simply use the old reference
;; net0 and net1 are unchanged - structural sharing via CHAMP
```

3.3 2.3 Core Data Structures (All Persistent)

3.3.1 2.3.1 Identity Types

```
;; In racket/prologos/propagator.rkt

;; Deterministic counters (no gensym - no global state)
(struct cell-id (n) #:transparent)
(struct prop-id (n) #:transparent)
```

Cell and propagator identities are monotonic counters *inside* the network. This makes networks deterministic (no gensym side effects) and serializable.

3.3.2 2.3.2 `prop-cell` — Propagator Cell (Immutable)

```
(struct prop-cell
  (value        ;; Expr - current lattice value (starts at bot)
   dependents)  ;; champ-root (set of prop-id → #t)
  #:transparent)
```

Note: no `id` field in the cell struct itself — the identity is the key in the network's cells map. No `domain` field — the merge function is stored in the network's `merge-fns` map, keyed by cell-id.

3.3.3 2.3.3 `propagator` — Monotone Function (Immutable)

```
(struct propagator
  (inputs      ;; list of cell-id
   outputs     ;; list of cell-id
   fire-fn)    ;; (prop-network → prop-network) - pure state transformer
  #:transparent)
```

The `fire-fn` is a **pure function** from network to network. It reads input cells from the network, computes new values, and returns a network with updated output cells. No side effects.

3.3.4 2.3.4 `prop-network` — The Network as Value

```
(struct prop-network
  (cells            ;; champ-root : cell-id → prop-cell
   propagators      ;; champ-root : prop-id → propagator
   worklist         ;; list of prop-id (ephemeral; empty at quiescence)
   next-cell-id     ;; Nat - monotonic counter for cell ids
   next-prop-id     ;; Nat - monotonic counter for propagator ids
   fuel             ;; Nat - step limit to prevent runaway
   contradiction    ;; #f | cell-id - first contradiction encountered
   merge-fns        ;; champ-root : cell-id → (Expr Expr → Expr)
   contradiction-fns) ;; champ-root : cell-id → (Expr → Bool), optional
  #:transparent)
```

Key design choices:

- **CellId = Nat counter**: deterministic, no gensym, serializable
- **merge-fns** stored per-cell: captures the lattice's `join` at cell creation

- **contradiction-fns** stored per-cell: optional predicate that detects if a cell value represents contradiction (`top`). This avoids requiring a `top?` method in the `Lattice` trait or introducing a `BoundedLattice` sub-trait (Prologos has **no trait hierarchies** — see DESIGN$_{\text{PRINCIPLES.org}}$). Instead, contradiction detection is a per-cell concern supplied at cell creation time.

- **worklist** is a plain Racket list: ephemeral within `run-to-quiescence`, always empty in saved snapshots (quiescent networks have empty worklists)

- **All CHAMP-backed**: O(log n) O(C) where C 7 for practical n

3.4 2.4 Pure Operations

All operations take a network and return a new network. The old network is never modified.

```
;; Create empty network
(define (make-prop-network [fuel 1000000])
  (prop-network champ-empty    ;; cells
                champ-empty    ;; propagators
                '()            ;; worklist
                0              ;; next-cell-id
                0              ;; next-prop-id
                fuel           ;; fuel
                #f             ;; no contradiction
                champ-empty    ;; merge-fns
                champ-empty))  ;; contradiction-fns

;; Add a new cell: returns (values new-network cell-id)
;; contradicts? is optional (#f = no contradiction detection at this cell)
(define (net-new-cell net initial-value merge-fn [contradicts? #f])
  (define id (cell-id (prop-network-next-cell-id net)))
  (define cell (prop-cell initial-value champ-empty))
  (define net*
    (struct-copy prop-network net
      [cells (champ-insert (prop-network-cells net)
                           (equal-hash-code id) id cell)]
      [merge-fns (champ-insert (prop-network-merge-fns net)
                               (equal-hash-code id) id merge-fn)]
```

```
      [next-cell-id (+ 1 (prop-network-next-cell-id net))]))
  (values
   (if contradicts?
       (struct-copy prop-network net*
         [contradiction-fns
          (champ-insert (prop-network-contradiction-fns net*)
                        (equal-hash-code id) id contradicts?)])
       net*)
   id))

;; Read a cell's value
(define (net-cell-read net cid)
  (define cell (champ-lookup (prop-network-cells net)
                              (equal-hash-code cid) cid))
  (if (eq? cell 'none)
      (error 'net-cell-read "unknown cell: ~a" cid)
      (prop-cell-value cell)))

;; Write a cell: computes merge-fn(old, new), enqueues dependents if changed
(define (net-cell-write net cid new-val)
  (define cells (prop-network-cells net))
  (define cell (champ-lookup cells (equal-hash-code cid) cid))
  (when (eq? cell 'none)
    (error 'net-cell-write "unknown cell: ~a" cid))
  (define merge-fn (champ-lookup (prop-network-merge-fns net)
                                 (equal-hash-code cid) cid))
  (define old-val (prop-cell-value cell))
  (define merged (merge-fn old-val new-val))
  (if (equal? merged old-val)
      net  ;; No change - return same network
      (let* ([new-cell (struct-copy prop-cell cell [value merged])]
             [new-cells (champ-insert cells (equal-hash-code cid) cid new-cell)]
             ;; Enqueue dependents
             [deps (champ-keys (prop-cell-dependents cell))]
             [new-wl (append deps (prop-network-worklist net))])
        (struct-copy prop-network net
          [cells new-cells]
          [worklist new-wl]))))

;; Add a propagator: returns (values new-network prop-id)
```

```
(define (net-add-propagator net input-ids output-ids fire-fn)
  (define pid (prop-id (prop-network-next-prop-id net)))
  (define prop (propagator input-ids output-ids fire-fn))
  ;; Register pid as dependent of each input cell
  (define new-cells
    (for/fold ([cells (prop-network-cells net)])
              ([cid (in-list input-ids)])
      (define cell (champ-lookup cells (equal-hash-code cid) cid))
      (if (eq? cell 'none) cells
          (let ([new-deps (champ-insert (prop-cell-dependents cell)
                                        (equal-hash-code pid) pid #t)])
            (champ-insert cells (equal-hash-code cid) cid
                          (struct-copy prop-cell cell [dependents new-deps])))))
  (values
   (struct-copy prop-network net
     [cells new-cells]
     [propagators (champ-insert (prop-network-propagators net)
                                (equal-hash-code pid) pid prop)]
     [next-prop-id (+ 1 (prop-network-next-prop-id net))]
     ;; Schedule initial firing
     [worklist (cons pid (prop-network-worklist net))])
   pid))
```

3.5 2.5 Concrete `fire-fn` Example: Adder Propagator

To ground the abstract `fire-fn` concept, here is a concrete example: a propagator that adds two cells and writes the result to a third.

```
;; adder-fire-fn : cell-id cell-id cell-id → (prop-network → prop-network)
;; Reads cells A and B, writes A+B to cell C
(define (make-adder-fire-fn a-id b-id c-id)
  (lambda (net)
    (define a-val (net-cell-read net a-id))
    (define b-val (net-cell-read net b-id))
    ;; Only propagate if both inputs are known (not bot)
    (if (and (not (eq? a-val 'bot)) (not (eq? b-val 'bot)))
        (net-cell-write net c-id (+ a-val b-val))
        net)))  ;; No change - return same network

;; Usage: wire up an adder network
```

```
(define net0 (make-prop-network))
(define-values (net1 cell-a) (net-new-cell net0 'bot flat-join))
(define-values (net2 cell-b) (net-new-cell net1 'bot flat-join))
(define-values (net3 cell-c) (net-new-cell net2 'bot flat-join))
(define-values (net4 adder-pid)
  (net-add-propagator net3
    (list cell-a cell-b)   ;; inputs
    (list cell-c)          ;; outputs
    (make-adder-fire-fn cell-a cell-b cell-c)))

;; Write to inputs, run to quiescence
(define net5 (net-cell-write net4 cell-a 3))
(define net6 (net-cell-write net5 cell-b 4))
(define net7 (run-to-quiescence net6))
(net-cell-read net7 cell-c) ;; → 7
```

Key observations:

- The `fire-fn` is a **closure** capturing cell-ids, returning a **pure function** from network to network
- It reads inputs via `net-cell-read` (from the network argument, not side effects)
- It writes via `net-cell-write` (returns new network, not mutation)
- If inputs are insufficient (`bot`), it returns the network unchanged (idempotent)
- The same `fire-fn` works correctly regardless of scheduling order

For the logic engine (Phase 7), a typical `fire-fn` will be a *clause propagator* that attempts unification and writes answer substitutions to a table cell. See Appendix A: Resolution by Example for a worked walkthrough.

3.6 2.6 `run-to-quiescence` — Pure Loop

The scheduler is not a separate struct — it is the `worklist` field of the network itself. `run-to-quiescence` is a pure tail-recursive function.

```
(define (run-to-quiescence net)
  (cond
    ;; Already contradicted - stop
```

```
[(prop-network-contradiction net) net]
;; Fuel exhausted - stop
[(<= (prop-network-fuel net) 0) net]
;; Worklist empty - quiescent (fixed point reached)
[(null? (prop-network-worklist net)) net]
;; Fire next propagator
[else
 (let* ([pid (car (prop-network-worklist net))]
        [rest (cdr (prop-network-worklist net))]
        [net* (struct-copy prop-network net
                [worklist rest]
                [fuel (sub1 (prop-network-fuel net))])]
        [prop (champ-lookup (prop-network-propagators net*)
                            (equal-hash-code pid) pid)])
   (if (eq? prop 'none)
       ;; Propagator removed or unknown - skip
       (run-to-quiescence net*)
       ;; Fire: pure function from network to network
       (run-to-quiescence ((propagator-fire-fn prop) net*))))]))
```

Properties:

- **Pure**: input is a `prop-network`, output is a `prop-network`
- **Deterministic**: same inputs → same fixed point (lattice commutativity)
- **Convergent**: finite-height lattices guarantee termination
- **Fuel-limited**: prevents runaway; configurable via `fuel` field
- **Backtrackable**: keep old `net` reference = instant O(1) backtrack

3.7 2.7 Contradiction Handling (Per-Cell `contradicts?` Predicate)

Contradiction is detected via a **per-cell predicate**, not a trait method. Each cell may optionally have a `contradicts?` function (supplied at creation time via `net-new-cell`) that checks if a value represents inconsistency.

This design avoids requiring `top?` in the `Lattice` trait (which would force every lattice to define it, even those without a sensible top element) and avoids introducing a `BoundedLattice` sub-trait (Prologos has no trait hierarchies — see DESIGN$_{\text{PRINCIPLES.org}}$ § "No Trait Hierarchies").

```
;; In net-cell-write, after computing merged:
(define (net-cell-write net cid new-val)
  ;; ... (merge as before) ...
  ;; After merge, check for contradiction:
  (define cfn (champ-lookup (prop-network-contradiction-fns net)
                            (equal-hash-code cid) cid))
  (define contradicted?
    (and (not (eq? cfn 'none))   ;; cell has a contradicts? fn
         (cfn merged)))          ;; the merged value is contradictory
  (if contradicted?
      (struct-copy prop-network net* [contradiction cid])
      net*))

;; Example contradiction predicates for standard lattices:
;; FlatLattice: (lambda (v) (and (some? v) (right? (unwrap v))))  ;; some(right(unit))
;; IntervalLattice: (lambda (v) (interval-empty? v))              ;; empty interval = c
;; SetLattice: #f  ;; no contradiction (sets only grow, never contradict)
```

At the pure-network level, contradiction simply records which cell was contradicted. The ATMS layer (Phase 5) interprets this and performs dependency-directed backtracking. Without ATMS, contradiction halts the network.

3.8 2.8 LVars Are Subsumed by Cells

A key simplification: **LVars are just cells in a persistent network**.

LVar operation	Network equivalent
`lvar-new`	`net-new-cell` with appropriate `merge-fn`
`lvar-put`	`net-cell-write` (already does join-on-write)
`lvar-get`	`net-cell-read` (synchronous in single-thread)
`lvar-freeze`	Read cell value from a quiescent network
`lvar-set-add`	`net-cell-write` on cell with `SetLattice`
`lvar-map-put`	`net-cell-write` on cell with `MapLattice`

This elimination removes the need for a separate LVar module and ~10 dedicated LVar AST nodes. The propagator network *is* the lattice-compatible collection — its cells natively do join-on-write via the `merge-fn`.

Threshold reads: for the synchronous logic engine (not parallel), threshold reads are implemented as propagators that check conditions after quiescence. True blocking threshold reads (for parallel LVar-style programming) are deferred to when actor/place integration is built.

3.9 2.9 AST Nodes (~12)

New `expr-*` structs in `syntax.rkt`:

Node	Fields	Semantics
`expr-prop-network`	net-value	Runtime wrapper for network
`expr-cell-id`	n	Cell identity (Nat counter)
`expr-net-new`	fuel-expr	Create empty network
`expr-net-new-cell`	net, init-val, merge	Add cell, return (net, id)
`expr-net-cell-read`	net, cell-id	Read cell value
`expr-net-cell-write`	net, cell-id, val	Write (merge) value
`expr-net-add-prop`	net, ins, outs, fn	Add propagator
`expr-net-run`	net	Run to quiescence
`expr-net-snapshot`	net	Snapshot (identity — free)
`expr-net-contradict?`	net	Check for contradiction
`expr-net-type`	—	Type constructor PropNetwork
`expr-cell-id-type`	—	Type constructor CellId

3.10 2.10 New/Modified Files

File	Changes
`racket/prologos/propagator.rkt`	NEW: Persistent network, pure ops
`racket/prologos/syntax.rkt`	+12 AST nodes
`racket/prologos/typing-core.rkt`	Infer/check rules for network ops
`racket/prologos/reduction.rkt`	Reduction rules for network ops
`racket/prologos/elaborator.rkt`	Surface → core for network ops
`racket/prologos/zonk.rkt`	Zonk cases for new nodes
`racket/prologos/pretty-print.rkt`	PP cases for new nodes
`racket/prologos/qtt.rkt`	Multiplicity for network ops
`racket/prologos/substitution.rkt`	Subst cases for new nodes
`racket/prologos/unify.rkt`	Unify cases for new nodes
`tests/test-propagator.rkt`	NEW
`tests/test-propagator-network.rkt`	NEW
`tests/test-propagator-quiescence.rkt`	NEW

3.11 2.11 Tests (~60)

- Network creation: empty, with fuel
- Cell creation: returns new network + cell-id

- Cell read: returns value
- Cell write: merge produces join (FlatLattice, SetLattice)
- Cell write: no change when value doesn't increase → same network returned
- Cell write: enqueues dependents
- Propagator: two cells connected, write propagates
- Propagator: diamond network (A → B, A → C, B+C → D)
- run-to-quiescence: fires to fixed point
- run-to-quiescence: fuel limit prevents runaway
- run-to-quiescence: contradiction recorded in network
- Persistence: old network unchanged after write
- Persistence: old network unchanged after run-to-quiescence
- Snapshot: identity operation (`net` is already persistent)
- Backtracking: use old reference after contradiction
- LVar-style: cell with SetLattice merge, add elements, freeze via read
- LVar-style: cell with MapLattice merge, pointwise join
- Integration: lattice instances work through network cells

3.12 2.12 Dependencies

- Phase 1 (Lattice trait and instances)
- `champ-insert-join` in `champ.rkt` (used internally for cell merge)
- Existing trait resolution for Lattice dictionary dispatch
- 12+ files modified (standard AST node pipeline)

—

4 Phase 3: PropNetwork as Prologos Type

4.1 3.1 Goal

Expose the Racket-level persistent PropNetwork to Prologos's type system. This phase adds the 12 AST nodes defined in Phase 2's design, threading them through the full 12-file pipeline (syntax → typing-core → reduction → elaborator → zonk → pretty-print → qtt → substitution → unify → tests).

4.2 3.2 Why a Separate Phase

Phase 2 implements the Racket-level data structures and algorithms. Phase 3 wires them into Prologos as first-class values. This separation follows the established pattern: Racket infrastructure first, then Prologos type system integration.

4.3 3.3 Type Signatures

```
;; PropNetwork is an opaque type wrapping the persistent network
deftype PropNetwork
  mk-prop-network : PropNetwork

;; CellId identifies a cell within a network
deftype CellId
  mk-cell-id : Nat -> CellId

;; Operations as typed functions
spec net-new : Nat -> PropNetwork
spec net-new-cell : {A : Type} where (Lattice A)
  PropNetwork -> A -> (A -> A -> A) -> [PropNetwork * CellId]
spec net-cell-read : {A : Type} PropNetwork -> CellId -> A
spec net-cell-write : {A : Type} PropNetwork -> CellId -> A -> PropNetwork
spec net-add-propagator : PropNetwork -> [List CellId] -> [List CellId]
  -> [PropNetwork -> PropNetwork] -> [PropNetwork * PropId]
spec net-run : PropNetwork -> PropNetwork
spec net-snapshot : PropNetwork -> PropNetwork
spec net-contradict? : PropNetwork -> Bool
```

4.4 3.4 LVar Operations as Library Functions

LVar-style operations are library functions on top of PropNetwork, not separate AST nodes:

```
;; A "growing set" = a cell with SetLattice merge
spec make-set-cell : {A : Type} where (Eq A)
  PropNetwork -> [PropNetwork * CellId]
defn make-set-cell [net]
  [net-new-cell net #{} set-union]

;; A "growing map" = a cell with MapLattice merge
spec make-map-cell : {K V : Type} where (Eq K) (Lattice V)
  PropNetwork -> [PropNetwork * CellId]
defn make-map-cell [net]
  [net-new-cell net {} [merge-with join]]
```

4.5 3.5 AST Nodes (12)

Same 12 nodes as Phase 2 design (`expr-prop-network`, `expr-cell-id`, `expr-net-new`, `expr-net-new-cell`, `expr-net-cell-read`, `expr-net-cell-write`, `expr-net-add-prop`, `expr-net-run`, `expr-net-snapshot`, `expr-net-contradict?`, `expr-net-type`, `expr-cell-id-type`).

4.6 3.6 New/Modified Files

File	Changes
`racket/prologos/syntax.rkt`	+12 AST nodes
`racket/prologos/typing-core.rkt`	Infer/check rules for network ops
`racket/prologos/reduction.rkt`	Reduction rules (delegate to .rkt)
`racket/prologos/elaborator.rkt`	Surface → core translation
`racket/prologos/zonk.rkt`	Zonk cases
`racket/prologos/pretty-print.rkt`	PP cases
`racket/prologos/qtt.rkt`	Multiplicity (:w for networks)
`racket/prologos/substitution.rkt`	Subst cases
`racket/prologos/unify.rkt`	Unify cases
`lib/prologos/core/propagator.prologos`	NEW: Library wrappers (set/map cells)
`tests/test-propagator-types.rkt`	NEW
`tests/test-propagator-integration.rkt`	NEW
`tests/test-propagator-lvar.rkt`	NEW: LVar-style cell tests

4.7 3.7 Tests (~50)

- Type checking: `net-new` returns `PropNetwork`
- Type checking: `net-cell-read` returns correct element type
- Type checking: `net-cell-write` returns `PropNetwork`
- Eval: create network, add cells, read values
- Eval: write value, verify merge
- Eval: propagator fires on cell write
- Eval: diamond network (A → B, A → C, B+C → D)
- Eval: run-to-quiescence reaches fixed point
- Eval: fuel exhaustion
- Eval: contradiction detection
- Persistence: old network unchanged after operations
- LVar-style: set cell grows via union
- LVar-style: map cell does pointwise join
- LVar-style: freeze = read from quiescent network
- Integration: Lattice trait resolution for cell merge functions

4.8 3.8 Dependencies

- Phase 2 (Racket-level PropNetwork implementation)
- Phase 1 (Lattice trait for type-level merge function constraints)

—

5 Phase 4: UnionFind — Persistent Disjoint Sets

5.1 4.1 Goal

Implement a persistent union-find data structure (Conchon & Filliâtre 2007) with backtracking support. This is the core data structure for unification in the logic engine. Unlike the current metavar store (mutable hash table), a persistent union-find supports efficient backtracking for search.

5.2 4.2 Design

The union-find uses path splitting (not path compression) to maintain persistence. Each node stores a parent reference and a rank. Find follows parent pointers to the root. Union merges by rank.

```
;; In racket/prologos/union-find.rkt

(struct uf-store
  (nodes)     ;; persistent map: id → uf-node
  #:transparent)

(struct uf-node
  (parent     ;; id (self = root)
   rank       ;; natural number
   value)     ;; optional payload (for unification: the term)
  #:transparent)

;; Create empty store
(define (uf-empty) (uf-store (hasheq)))

;; Make a new set with one element
(define (uf-make-set store id value)
  (uf-store
   (hash-set (uf-store-nodes store) id
             (uf-node id 0 value))))

;; Find root of id (with path splitting for amortized cost)
(define (uf-find store id)
  ;; Returns (values root-id updated-store)
  ;; Path splitting: point each node to its grandparent
  ...)

;; Union two sets, merge by rank
(define (uf-union store id1 id2 merge-fn)
  ;; Find roots, merge smaller into larger
  ;; merge-fn combines values if both have payload
  ;; Returns updated store
  ...)

;; Snapshot for backtracking
```

```
(define (uf-snapshot store) store)  ;; Already persistent!
```

5.3 4.3 Key Properties

- **Persistent**: Union/find return new stores, old stores unchanged
- **Backtrackable**: Save a reference to the old store = instant backtrack
- **Efficient**: O(log n) find and union (path splitting without compression)
- **Value-carrying**: Nodes carry optional payloads (unified terms)

5.4 4.4 Integration with Logic Engine: UF vs Cell Division of Labor

The union-find and propagator cells serve **complementary** roles in the logic engine. They are not alternatives — both are needed:

Operation	Uses UF	Uses PropNetwork Cell
`?x = ?y` (var-var)	`uf-union x y`	—
`?x = 42` (var-value)	`uf-find x` → set value	—
`?x = f(?y, ?z)` (complex)	`uf-union` + subterm cells	Cell per subterm for propagation
Lattice-valued accumulation	—	`net-cell-write` with join
Table answer sets	—	Cell with `SetLattice` merge
Numeric constraints	—	Cell with `IntervalLattice`
Backtracking	Keep old `uf-store`	Keep old `prop-network`

The division of labor:

- **UnionFind**: structural equality — discovering that two variables must refer to the same term. This is the fast path for standard Prolog-style unification. O(log n) find-with-path-splitting.
- **PropNetwork cells**: lattice-valued accumulation — aggregating information that can only grow (sets of answers, numeric intervals, constraint domains). This is the slow-but-general path for constraint propagation.

In practice, a logic engine query creates *both*:

1. A `uf-store` for the substitution (logic variable bindings)

2. A `prop-network` for constraint cells (tables, aggregates, numeric domains)

Both are persistent, both support O(1) backtracking. The solver threads both through computation as a pair: `(uf-store, prop-network)`.

5.5 4.5 AST Nodes (~6)

Node	Fields	Semantics
`expr-uf-empty`	—	Create empty union-find
`expr-uf-make-set`	store, id, val	Add new singleton set
`expr-uf-find`	store, id	Find root of set
`expr-uf-union`	store, id1, id2	Union two sets
`expr-uf-value`	store, id	Get value at id's root
`expr-uf-type`	—	Type constructor UnionFind

5.6 4.6 New/Modified Files

File	Changes
`racket/prologos/union-find.rkt`	NEW: Persistent UF
`racket/prologos/syntax.rkt`	+6 AST nodes
+ standard pipeline files	Type rules, reduction, etc.
`tests/test-union-find.rkt`	NEW

5.7 4.7 Tests (~30)

- Empty store creation
- Make-set: adds node
- Find: returns self for singleton
- Union: merges two sets
- Union: rank-based merging
- Find after union: both ids → same root
- Value: retrieve payload after union
- Persistence: old store unchanged after union
- Backtracking: restore snapshot works

- Performance: 1000 unions, find all roots
- Integration: as substitution store for simple unification

5.8 4.8 Dependencies

- None (self-contained data structure)
- Can proceed in parallel with Phase 3

—

6 Phase 5: Persistent ATMS Layer — Hypothetical Reasoning

6.1 5.1 Goal

Implement the Assumption-Based Truth Maintenance System (ATMS) as a **persistent, immutable value**. Like the propagator network, the ATMS is backed entirely by CHAMP maps. Backtracking = use old reference. Switching worldviews = `struct-copy` with new `believed` set.

This validates the "Multiverse Mechanism" from the propagator research: choice-point forking maps directly onto ATMS worldview management.

6.2 5.2 Core Data Structures (All Persistent)

6.2.1 `assumption` — Hypothetical Premise

```
;; In racket/prologos/atms.rkt

(struct assumption-id (n) #:transparent)

(struct assumption
  (name       ;; symbol (for display)
   datum)     ;; optional: the value this assumption asserts
  #:transparent)
```

6.2.2 `supported-value` — Value + Justification

```
(struct supported-value
  (value      ;; the lattice value
   support)   ;; champ-root : assumption-id → #t (set of assumptions)
  #:transparent)
```

6.2.3 `tms-cell` — Truth-Maintained Cell (Immutable)

A TMS cell holds multiple contingent values, each justified by a different assumption set. It generalizes a prop-cell to handle nondeterminism.

```
(struct tms-cell
  (values        ;; list of supported-value
   dependents)   ;; champ-root (set of prop-id → #t)
  #:transparent)

;; Note: immutable. Update returns new tms-cell via struct-copy.
;; The cell identity (tms-cell-id) is the key in the ATMS's cells map.
```

6.2.4 `atms` — The Persistent ATMS

```
(struct atms
  (network         ;; prop-network (the underlying persistent network)
   assumptions     ;; champ-root : assumption-id → assumption
   nogoods         ;; list of champ-root (each a set of assumption-ids → #t)
   tms-cells       ;; champ-root : cell-id → tms-cell
   next-assumption ;; Nat - monotonic counter
   believed)       ;; champ-root : assumption-id → #t (current worldview)
  #:transparent)
```

Key design choices:

- **All CHAMP-backed**: assumptions, nogoods (as frozen sets), tms-cells, believed
- **Worldview switching**: `(struct-copy atms a [believed new-set])` — O(1)
- **Backtracking**: keep old `atms` reference — O(1), structural sharing
- **No mutable state**: all operations return new `atms` values

6.3 5.3 Pure Operations

```
;; Create a new assumption: returns (values new-atms assumption-id)
(define (atms-assume atms name datum)
  (define aid (assumption-id (atms-next-assumption atms)))
  (define a (assumption name datum))
  (values
```

```
  (struct-copy atms atms
    [assumptions (champ-insert (atms-assumptions atms)
                               (equal-hash-code aid) aid a)]
    [next-assumption (+ 1 (atms-next-assumption atms))]
    [believed (champ-insert (atms-believed atms)
                            (equal-hash-code aid) aid #t)])
  aid))

;; Record a nogood: returns new atms with pruned worldview
(define (atms-add-nogood atms nogood-set)
  (struct-copy atms atms
    [nogoods (cons nogood-set (atms-nogoods atms))]))

;; Check consistency of an assumption set
(define (atms-consistent? atms assumption-set)
  (not (for/or ([ng (in-list (atms-nogoods atms))])
         (champ-subset? ng assumption-set))))

;; Switch worldview: returns new atms with different believed set
(define (atms-with-worldview atms new-believed)
  (struct-copy atms atms [believed new-believed]))
```

6.4 5.4 The amb Operator (Pure)

amb creates a choice point with n alternatives. Returns new ATMS.

```
(define (atms-amb atms alternatives)
  ;; 1. Create fresh hypotheses h1..hn
  (define-values (atms* hyps)
    (for/fold ([a atms] [hs '()])
              ([alt (in-list alternatives)]
               [i (in-naturals)])
      (define-values (a* hid) (atms-assume a (format "h~a" i) alt))
      (values a* (cons hid hs))))
  ;; 2. Record mutual exclusion: any pair is a nogood
  (define atms**
    (for*/fold ([a atms*])
               ([h1 (in-list hyps)]
                [h2 (in-list hyps)]
                #:when (not (equal? h1 h2)))
```

```
      (atms-add-nogood a (champ-insert (champ-insert champ-empty
                                          (equal-hash-code h1) h1 #t)
                                        (equal-hash-code h2) h2 #t))))
  ;; 3. Create TMS cell with supported values
  (values atms** hyps))
```

6.5 5.5 Two-Tier Mode: Lazy ATMS Activation

The ATMS adds overhead for every cell operation: label management, support set tracking, consistency checking. For **deterministic** queries (no `amb`, no choice points), this overhead is pure waste.

Design requirement: the ATMS operates in two tiers:

Tier 1 — PropNetwork Only (default):

- No ATMS overhead. Just a `prop-network` with pure propagation.
- Cells hold simple values, not supported values.
- `run-to-quiescence` operates directly on the persistent network.
- This is sufficient for Datalog evaluation, deterministic constraint propagation, and simple tabled predicates.

Tier 2 — Full ATMS (activated on first `amb`):

- When the solver encounters its first choice point (`amb`), it upgrades to full ATMS mode by wrapping the current network in an `atms` struct.
- From that point on, all cell values become supported values (tagged with assumption sets).
- Nogoods, worldview switching, and DDB become available.

```
;; The solver state is either a bare network or a full ATMS
;; (struct solver-state (kind data) #:transparent)
;; kind is 'network | 'atms

;; On first amb:
(define (solver-upgrade-to-atms state)
  (if (eq? (solver-state-kind state) 'atms)
      state  ;; already upgraded
      (solver-state 'atms
        (atms-from-network (solver-state-data state)))))
```

```
;; atms-from-network : prop-network → atms
;; Wraps existing network, converting all cell values to
;; unconditionally supported values (support =  = always true)
```

The `solve-with` configuration supports:

- `:strategy :auto` (default) — starts in Tier 1, upgrades on first `amb`
- `:strategy :depth-first` — pure PropNetwork, no ATMS, chronological choice
- `:strategy :atms` — full ATMS from the start

This addresses the research doc's §14.1 concern: "For simple deterministic programs, the ATMS layer is pure cost." Lazy activation means deterministic programs never pay ATMS overhead.

6.6 5.6 Contradiction Handler (Dependency-Directed Backtracking)

When the underlying prop-network detects contradiction (a cell's merged value = top):

1. Extract the support set from the contradicted cell's TMS values
2. Record as a nogood in the ATMS
3. The nogood automatically prunes that worldview from future exploration
4. Return new ATMS with nogood recorded

This is pure: each step returns a new `atms` value. Dependency-directed backtracking identifies *which* choice was wrong, not just the most recent.

6.7 5.7 Answer Collection (Pure)

```
;; Enumerate all consistent worldviews and extract answers
(define (atms-solve-all atms goal-cell-id)
  ;; 1. Enumerate maximal consistent assumption sets
  ;; 2. For each worldview, run the network to quiescence
  ;; 3. Extract goal cell value under that worldview
  ;; 4. Collect distinct answers
  ;; Returns: list of answer values
  ...)
```

6.8 5.8 AST Nodes (~10)

Node	Fields	Semantics
`expr-atms-new`	network	Create ATMS wrapping network
`expr-amb`	atms, alternatives	Create choice point
`expr-assume`	atms, name, datum	Create hypothesis
`expr-retract`	atms, assumption-id	Retract hypothesis
`expr-nogood`	atms, assumption-set	Record nogood
`expr-solve-all`	atms, goal	Collect all solutions
`expr-tms-read`	atms, cell, worldview	Read under worldview
`expr-tms-write`	atms, cell, val, support	Write supported value
`expr-atms-type`	—	Type constructor ATMS
`expr-supported-type`	val-type	Type of SupportedValue

6.9 5.9 New/Modified Files

File	Changes
`racket/prologos/atms.rkt`	NEW: Persistent ATMS
`racket/prologos/syntax.rkt`	+10 AST nodes
`racket/prologos/propagator.rkt`	Contradiction → record in ATMS
+ standard pipeline files	Type rules, reduction, etc.
`tests/test-atms.rkt`	NEW
`tests/test-atms-search.rkt`	NEW
`tests/test-atms-backtrack.rkt`	NEW

6.10 5.10 Tests (~50)

- ATMS creation from network
- Assumption creation: returns new atms + id
- Supported value: value + support set
- TMS cell: multiple contingent values
- Consistency check: no nogood subsets
- amb: creates mutual exclusion nogoods
- amb: different alternatives under different worldviews
- Nogood recording: prunes invalid worldviews

- Dependency-directed backtracking: contradiction identifies cause
- Worldview switching: `atms-with-worldview` — O(1)
- Backtracking: keep old atms reference — O(1)
- Persistence: old atms unchanged after assume/nogood
- solve-all: enumerates all consistent answers
- solve-all: deduplicates answers
- Integration: persistent network + persistent ATMS
- Performance: 100-alternative amb, solve-all

6.11 5.11 Dependencies

- Phase 2 (Persistent PropNetwork — ATMS wraps a network)
- Phase 3 (PropNetwork as Prologos type — for typed ATMS operations)

—

7 Phase 6: Tabling — SLG-Style Memoization

7.1 6.1 Goal

Implement tabling for completeness. Without tabling, left-recursive rules cause infinite loops. Tabling memoizes intermediate results and detects fixed-point completion.

7.2 6.2 Design (XSB-Style SLG Resolution)

Concept	Implementation
Table	PropNetwork cell with SetLattice merge
Producer	Propagator computing new answers for table cell
Consumer	Propagator reading from table cell
Completion	Table cell quiescent when no new answers propagate
Answer mode	`all` (collect all) or `lattice` (join)

7.3 6.3 Table Lifecycle

```
1. First call to tabled predicate → create table cell (SetLattice of answers)
2. Launch producer propagator → evaluates rules, writes answers to table cell
3. Recursive call to same predicate → create consumer propagator
   - Consumer reads from table cell
   - Re-fires when table cell's value grows (new answers appear)
4. Network reaches quiescence → no new answers propagate
5. Table cell value is the complete answer set
6. Subsequent calls → read from quiescent network (already computed)
```

7.4 6.4 Core Data Structures (Persistent)

The `table-store` is an **index**, not a store. It maps predicate call patterns to cell-ids in the PropNetwork. The actual answer data lives in PropNetwork cells with `SetLattice` merge — the table-store merely tells you *which* cell to look in.

```
;; In racket/prologos/tabling.rkt

(struct table-entry
  (predicate-name  ;; symbol
   call-pattern    ;; the specific instantiation pattern
   cell-id         ;; cell-id in the prop-network (answers live here)
   status)         ;; 'active | 'complete
  #:transparent)   ;; immutable - update via struct-copy

;; NOTE: no `answers` field - answers are stored in the PropNetwork cell
;; identified by `cell-id`. The cell uses SetLattice merge, so writing
;; a new answer to it computes set-union with existing answers.

(struct table-store
  (tables)     ;; champ-root : table-key → table-entry
  #:transparent)

;; Table key: predicate name + call pattern (up to renaming)
(define (table-key name args)
  (cons name (abstract-call-pattern args)))

;; Tables are backed by CHAMP for structural sharing.
;; Backtracking = keep old table-store reference + old prop-network.
```

```
;;
;; The lifecycle:
;; 1. First call → create table-entry + new cell in network
;; 2. Producer propagator writes answers to the cell
;; 3. Consumer propagators read from the cell (re-fire when it grows)
;; 4. Quiescence → mark table-entry status as 'complete
;; 5. Subsequent calls → read from the cell (already computed)
```

7.5 6.5 Lattice Answer Modes

Following XSB Prolog:

- `all`: Table stores set of all distinct answer substitutions (`SetLattice` on substitutions)
- `lattice f`: Table stores lattice join of all answers via `f` (single aggregated value, new answers only "new" if they improve)
- `first`: Table frozen after first answer (`once` semantics)

7.6 6.6 Spec Metadata Integration

Tabling is declared via spec metadata:

```
spec ancestor : String -> String -> Prop
  :tabled true
  :answer-mode all        ;; default
```

This requires adding `:tabled` and `:answer-mode` cases to `parse-spec-metadata` in `macros.rkt` (following the `:examples` pattern).

7.7 6.7 AST Nodes (~8)

Node	Fields	Semantics
`expr-table-lookup`	name, args	Check if answer in table
`expr-table-new`	name, answer-mode	Create new table
`expr-table-add`	table, answer	Add answer to table
`expr-table-freeze`	table	Freeze table
`expr-table-answers`	table	Get all answers
`expr-table-complete?`	table	Check if table is complete
`expr-tabled`	spec-name	Mark predicate as tabled
`expr-table-type`	answer-type	Type of table

7.8 6.8 New/Modified Files

File	Changes
`racket/prologos/tabling.rkt`	NEW: Table store, lifecycle
`racket/prologos/macros.rkt`	:tabled, :answer-mode metadata
`racket/prologos/syntax.rkt`	+8 AST nodes
+ standard pipeline files	Type rules, reduction, etc.
`tests/test-tabling.rkt`	NEW
`tests/test-tabling-lattice.rkt`	NEW

7.9 6.9 Tests (~40)

- Table creation: cell with SetLattice merge in PropNetwork
- Table answer insertion: cell write grows answer set
- Network quiescence: table complete when no new answers propagate
- Producer: generates answers from rules, writes to table cell
- Consumer: reads from table cell, re-fires on growth
- Left-recursion: tabling terminates (the critical test)
- Lattice answer mode: min aggregation
- Lattice answer mode: set union aggregation
- First answer mode: stops after one
- Multiple tables: independent cells in same network
- Backtracking: persistent network, old table state preserved
- Spec metadata: :tabled parsed and stored
- Integration: tabling + persistent ATMS for recursive search

7.10 6.10 Dependencies

- Phase 3 (PropNetwork cells — tables are cells with SetLattice merge)
- Phase 5 (ATMS — tabling + nondeterminism interact)

—

8 Phase 7: Surface Syntax — `defr`, `rel`, `solve`, `&>`

8.1 7.1 Goal

Implement the user-facing relational language as described in RELATIONAL$_{\text{LANGUAGEVISION.org}}$.

8.2 7.2 Reader Changes

The reader must handle:

Syntax	Reader Output
`?var`	`(logic-var var)`
`-var`	`(mode-var in var)`
`+var`	`(mode-var out var)`
`&>`	`($clause-sep)`
`(goal ...)`	`(goal ...)` (parenthetical = relational)

8.3 7.3 Parser Changes

New surface AST nodes:

Form	Surface AST
`defr name [args] body`	`(surf-defr name args clauses)`
`(rel [args] body)`	`(surf-rel args clauses)`
`&> g1 g2 ...`	`(surf-clause (g1 g2 ...))` within defr/rel
`(solve [goal])`	`(surf-solve goal)`
`(solve-with :opts goal)`	`(surf-solve-with opts goal)`
(?x ?y)=	`(surf-unify x y)`
`(is ?x [expr])`	`(surf-is var expr)`

8.4 7.4 Elaboration

Relations elaborate to propagator networks. Here is the concrete Racket-level translation for both `defr` and `solve`:

8.4.1 7.4.1 `defr` Elaboration

```
;; defr ancestor [?x ?y]
;;   &> (parent ?x ?y)
;;   &> (parent ?x ?z) (ancestor ?z ?y)
;;
```

```
;; elaborates to:
;; 1. Create table for ancestor (tabled by default)
;; 2. For each clause, create a propagator that:
;;    a. Unifies arguments with clause head
;;    b. Resolves subgoals (recursively, via table)
;;    c. If all succeed, adds answer to table
;; 3. Register in relation store
```

8.4.2 7.4.2 `solve` Elaboration (Functional-Relational Bridge)

The `solve` form is the bridge from relational goals back to functional values. Here is how it elaborates at the Racket level:

```
;; Source: functional code using solve
defn find-ancestors [person]
  (solve [ancestor person ?who])
;; Returns: Seq (Map Symbol Value)
```

Elaborates to:

```
;; Racket-level translation of (solve [ancestor person ?who])
(define (solve-ancestor-query person)
  (let* (;; 1. Create solver state (network + uf-store)
         [net0 (make-prop-network)]
         [uf0  (uf-empty)]

         ;; 2. Create logic variable cells
         ;;    ?who is the query variable; person is ground (bound)
         [net1+who (net-new-cell net0 'bot flat-join flat-top?)]
         [net1 (car net1+who)]
         [who-id (cdr net1+who)]

         ;; 3. Look up 'ancestor in the relation registry
         ;;    (registered by defr elaboration)
         [ancestor-rel (relation-lookup 'ancestor)]

         ;; 4. Create query propagators:
         ;;    - Instantiate ancestor's clauses with (person, ?who)
         ;;    - Wire table cell for ancestor
         [net2 (relation-instantiate ancestor-rel net1 uf0
                  (list (ground-val person) who-id))]
```

```
        ;; 5. Run to quiescence
        [net3 (run-to-quiescence net2)]

        ;; 6. Extract answers from the table cell
        [answers (table-answers net3 'ancestor)]

        ;; 7. Project query variable bindings into Maps
        [results (for/list ([subst (in-set answers)])
                   (hash 'who (substitution-lookup subst who-id)))])
  ;; Return as lazy sequence of maps
  (list->lseq results)))
```

Key points:

- `solve` creates a fresh `(network, uf-store)` pair scoped to the query
- Ground arguments are injected directly; logic variables become cells
- The relation's propagators run to quiescence in the fresh network
- Results are projected from table cells into `Seq (Map Symbol Value)`
- The entire operation is **pure** — no side effects, no global state

8.5 7.5 Grammar Updates

Both grammar files must be updated:

- `docs/spec/grammar.ebnf` — EBNF production rules
- `docs/spec/grammar.org` — Prose companion with examples

New productions:

```
relation-def  = "defr" , identifier , param-list , clause+ ;
anonymous-rel = "(" , "rel" , param-list , clause+ , ")" ;
clause        = "&>" , goal+ ;
goal          = "(" , goal-head , goal-arg* , ")" ;
goal-head     = identifier | "=" | "is" | "not" ;
goal-arg      = logic-var | expression ;
logic-var     = "?" , identifier ;
mode-var      = ( "-" | "+" ) , identifier ;
solve-expr    = "(" , "solve" , "[" , goal , "]" , ")" ;
solve-with    = "(" , "solve-with" , metadata* , "[" , goal , "]" , ")" ;
```

8.6 7.6 Compile-Time Stratification Check

Any program using `not` (negation-as-failure) must pass a stratification check at compile time. This is the minimum viable safety guarantee for negation:

1. **Build the predicate dependency graph**: For each `defr`, record which predicates it calls positively and negatively (via `not`).
2. **Compute SCCs**: Strongly connected components via Tarjan's algorithm. Each SCC is a group of mutually recursive predicates.
3. **Classify negative edges**: An edge `A --not--> B` is:
   - **Safe** if `B` is in a *lower* SCC than `A` (B is fully computed before A uses its negation)
   - **Unsafe** if `B` is in the *same* SCC as `A` (cyclic negation — the truth of `not B` depends on `A` which depends on `not B`)
4. **Reject unstratifiable programs**: If any negative edge is within an SCC, emit a compile-time error: "Cyclic negation detected: `A` and `B` are mutually recursive through negation. Consider restructuring."
5. **Assign strata**: Each SCC gets a stratum number (topological order of the condensation DAG). Lower strata are evaluated first; their results are frozen before higher strata use them via `not`.

```
;; In racket/prologos/stratify.rkt (NEW)

;; stratify : (Listof defr-info) → (Listof (Listof defr-info))
;; Returns a list of strata (each stratum = list of defrs to evaluate together)
;; Raises compile-time error if program is unstratifiable

(define (stratify defr-infos)
  (define dep-graph (build-dependency-graph defr-infos))
  (define sccs (tarjan-scc dep-graph))
  (define condensation (condense dep-graph sccs))
  ;; Check: no negative edge within any SCC
  (for ([scc (in-list sccs)])
    (for ([pred (in-list scc)])
      (for ([neg-dep (in-list (negative-dependencies pred))])
        (when (member neg-dep scc)
          (raise-stratification-error pred neg-dep scc)))))
```

```
;; Return strata in topological order
(topological-sort condensation))
```

This check runs during elaboration of `defr` forms. Programs without `not` are trivially stratifiable (single stratum) and incur no analysis cost.

8.7 7.7 AST Nodes (~15)

Node	Fields	Semantics
`expr-defr`	name, params, clauses	Named relation
`expr-rel`	params, clauses	Anonymous relation
`expr-clause`	goals	Single clause (disjunct)
`expr-goal-app`	name, args	Relational goal application
`expr-logic-var`	name, mode	Logic variable
`expr-unify-goal`	lhs, rhs	Unification goal (= ?x ?y)
`expr-is-goal`	var, expr	Functional evaluation in relation
`expr-not-goal`	goal	Negation-as-failure
`expr-solve`	goal	Solve returning Seq Map
`expr-solve-with`	opts, goal	Parameterized solve
`expr-solve-one`	goal	Solve returning first answer
`expr-relation-type`	param-types	Type of a relation
`expr-goal-type`	—	Type of a goal
`expr-cut`	—	Committed choice (once)
`expr-guard`	condition, goal	Guard evaluation

8.8 7.8 New/Modified Files

File	Changes
`racket/prologos/reader.rkt`	?var, -var, +var, &> handling
`racket/prologos/surface-syntax.rkt`	New surface AST structs
`racket/prologos/parser.rkt`	Parse defr, rel, solve, &>
`racket/prologos/macros.rkt`	process-defr, process-rel
`racket/prologos/elaborator.rkt`	Elaborate relations → propagators
`racket/prologos/stratify.rkt`	NEW: SCC + stratification check
`racket/prologos/syntax.rkt`	+15 AST nodes
`racket/prologos/typing-core.rkt`	Type rules for relational forms
+ standard pipeline files	All 12+ consuming modules
`docs/spec/grammar.ebnf`	New productions
`docs/spec/grammar.org`	New sections
`lib/prologos/core/relations.prologos`	NEW: Standard relations
`tests/test-relations-basic.rkt`	NEW
`tests/test-relations-tabling.rkt`	NEW
`tests/test-relations-search.rkt`	NEW
`tests/test-relations-ws.rkt`	NEW: WS-mode integration
`tests/test-solve.rkt`	NEW

8.9 7.9 Tests (~80)

- Reader: ?var, -var, +var parsed correctly
- Parser: defr, rel, &>, solve parsed correctly
- WS-mode: defr with indentation-based clauses
- Sexp-mode: defr with explicit (defr ...) form
- Basic relation: parent facts, query
- Recursive relation: ancestor with tabling
- Unification goal: = ?x ?y
- Functional evaluation: is ?x [expr]
- Multiple clauses: &> separator
- Anonymous relation: (rel [?x ?y] &> ...)
- Solve: returns Seq of substitution maps
- Solve: empty result for unsatisfiable goal

- Solve-with: :strategy option
- Solve-with: :timeout option
- Negation: not (goal)
- Mode annotations: -var, +var optimization hints
- Integration with functional code: solve in defn body
- Integration with traits: relation using trait methods
- Stratification: non-negated program = single stratum (trivial)
- Stratification: negated program with valid stratification compiles
- Stratification: cyclic negation detected and rejected at compile time
- Error messages: undefined relation, arity mismatch
- Performance: 100-fact database, recursive query

8.10 7.10 Dependencies

- ALL previous phases (Lattice, Cells, LVars, UF, ATMS, Tabling)

—

9 Phase Summary

Phase	Name	New AST	New Files	New Tests	Deps	Size
1	Lattice + champ-insert-join	0	3	25	None	Small
2	Persistent PropNetwork	0	3	60	Ph 1	Medium
3	PropNetwork Prologos Type	12	3	50	Ph 2	Medium
4	UnionFind	6	2	30	None (3)	Small
5	Persistent ATMS	10	3	50	Ph 3	Large
6	Tabling	8	3	40	Ph 3,5	Medium
7	Surface Syntax	15	11+	83	All	Large
TOTAL		51	28+	338		

Estimated total: ~51 new AST nodes (was 61 — LVar nodes eliminated), ~28+ new files, ~338 new tests.

Key changes from the original design:

- **Persistent architecture**: All structs are `#:transparent` (not `#:mutable`)
- **LVars subsumed**: 10 LVar AST nodes eliminated (LVars = network cells)
- **Phase 2 split**: Racket-level implementation (Phase 2) separated from Prologos type exposure (Phase 3)
- **Free backtracking**: O(1) via structural sharing (CHAMP maps)
- **Free snapshots**: Networks are values — keeping a reference IS a snapshot
- **Per-cell contradiction detection**: `contradicts?` predicate per cell, not in Lattice trait (no trait hierarchies — see DESIGN$_{\text{PRINCIPLES.org}}$)
- **ATMS two-tier mode**: Lazy activation on first `amb` — deterministic programs never pay ATMS overhead
- **Compile-time stratification**: SCC + negative edge check rejects unstratifiable programs at compile time

—

10 Interaction with Existing Infrastructure

10.1 Metavar System

The current metavar system (`current-meta-store`, `save/restore-meta-state!`) is *not* replaced by Phase 2's propagator network. The elaborator continues to use its existing metavar system for type inference. The propagator network is a *separate, parallel* system used by the logic engine at runtime.

The persistent PropNetwork architecture was directly motivated by the `save/restore-meta-state!` problem: the metavar store requires O(n) deep copies for speculative type checking, and restore requires explicit undo. The persistent approach avoids this entirely — backtracking is O(1) (keep old reference). In the future (post-Phase 7), the elaborator's metavar system could be refactored to use propagator cells internally.

10.2 Trait System

The `Lattice` trait (Phase 1) uses the existing trait infrastructure with no modifications. Lattice instances resolve via `resolve-trait-constraints!` like any other trait.

10.3 Spec Metadata

Phase 6 adds `:tabled` and `:answer-mode` to `parse-spec-metadata`. This follows the same pattern as `:examples` (Stage C of Extended Spec Hardening): explicit case in the metadata parser using `collect-constraint-values` or direct value capture.

10.4 Warnings

The logic engine may emit new warning types (e.g., "tabled predicate exceeded table size limit", "negation in unstratifiable position"). These follow the `warnings.rkt` pattern: new struct + parameter + emit/format.

10.5 QTT / Multiplicities

Logic variables live at multiplicity `:w` (unrestricted). They are shared across multiple goals and clauses. The binding environment (the substitution store) is functional/persistent and does not consume resources linearly.

Open question: should cells be linear (`:1`)? A cell created and consumed exactly once (write, then read, then discard) could be linear. For now, cells are unrestricted (`:w`).

10.6 Collections

The entire propagator network is backed by CHAMP maps (`champ.rkt`):

- `cells` map: `cell-id → prop-cell`
- `propagators` map: `prop-id → propagator`
- `merge-fns` map: `cell-id → merge-fn`
- ATMS `assumptions`, `tms-cells`, `believed`: all CHAMP maps
- Table stores: CHAMP map of table entries

The `champ-insert-join` helper (added in Phase 1) enables lattice-aware insert-with-merge at the CHAMP level. SetLattice cells use `set-union` as their merge function. MapLattice cells use `merge-with join`.

No separate LVar-Set or LVar-Map types are needed — these are simply cells in a PropNetwork with appropriate merge functions.

—

11 Appendix A: Resolution by Example — `ancestor` as Propagators

This appendix shows the complete elaboration of a `defr` definition into propagator network operations. It bridges the gap between the surface syntax (Phase 7) and the propagator substrate (Phases 2-6).

11.1 A.1 Source Program

```
;; Facts
defr parent [?x ?y]
  &> (= ?x "alice") (= ?y "bob")
  &> (= ?x "bob") (= ?y "carol")
  &> (= ?x "bob") (= ?y "dave")

;; Recursive relation (tabled by default)
defr ancestor [?x ?y]
  &> (parent ?x ?y)                        ;; clause 1: base case
  &> (parent ?x ?z) (ancestor ?z ?y)    ;; clause 2: recursive
```

11.2 A.2 Step 1: Table Creation

`ancestor` is tabled by default. The elaborator:

1. Creates a table entry in the `table-store` index: `(table-entry 'ancestor <call-pattern> cell-42 'active)`
2. Creates a PropNetwork cell for the table's answer set: `(net-new-cell net 'empty-set set-union)` → `cell-42`
3. The cell's merge function is `set-union` (`SetLattice`) — answers accumulate.

11.3 A.3 Step 2: Clause 1 → Producer Propagator

Clause 1: `&> (parent ?x ?y)` (base case)

The elaborator creates a propagator that:

```
;; Clause-1 fire-fn for ancestor(?x, ?y):
;;   For each parent fact matching (?x, ?y), write to ancestor's table cell
(define (make-ancestor-clause1-fire-fn table-cell-id)
  (lambda (net)
```

```
  ;; For each fact in parent's table:
  (define parent-answers
    (net-cell-read net parent-table-cell-id))
  ;; Each parent answer is an ancestor answer (base case)
  ;; Write each to the ancestor table cell
  (for/fold ([net* net])
            ([ans (in-set parent-answers)])
    (net-cell-write net* table-cell-id
                    (set-add (set) ans)))))
```

Inputs: `parent`'s table cell. Outputs: `ancestor`'s table cell.

11.4 A.4 Step 3: Clause 2 → Producer Propagator

Clause 2: `&> (parent ?x ?z) (ancestor ?z ?y)` (recursive case)

This creates a propagator that:

```
;; Clause-2 fire-fn for ancestor(?x, ?y):
;;   Join parent(?x, ?z) with ancestor(?z, ?y) → ancestor(?x, ?y)
(define (make-ancestor-clause2-fire-fn table-cell-id)
  (lambda (net)
    (define parent-facts (net-cell-read net parent-table-cell-id))
    (define ancestor-facts (net-cell-read net table-cell-id))
    ;; For each parent(x, z) and ancestor(z, y), derive ancestor(x, y)
    (define new-answers
      (for*/set ([p (in-set parent-facts)]
                 [a (in-set ancestor-facts)]
                 #:when (equal? (subst-lookup p '?y)    ;; z in parent
                                (subst-lookup a '?x)))  ;; z in ancestor
        (make-subst '?x (subst-lookup p '?x)
                    '?y (subst-lookup a '?y))))
    (net-cell-write net table-cell-id new-answers)))
```

Inputs: `parent`'s table cell AND `ancestor`'s table cell (self-reference!). Outputs: `ancestor`'s table cell.

The self-reference is what makes tabling essential: the propagator reads from the cell it writes to. Tabling ensures this reaches a fixed point rather than looping infinitely.

11.5 A.5 Step 4: Run to Quiescence

```
Iteration 1:
  parent table cell = {(alice,bob), (bob,carol), (bob,dave)}
  clause-1 fires → ancestor cell = {(alice,bob), (bob,carol), (bob,dave)}
  clause-2 fires → join parent × ancestor:
    parent(alice,bob)  ancestor(bob,carol) → ancestor(alice,carol)
    parent(alice,bob)  ancestor(bob,dave) → ancestor(alice,dave)
  ancestor cell = {(alice,carol), (alice,dave)}

Iteration 2:
  clause-2 fires again (ancestor cell grew):
    parent(alice,bob)  ancestor(bob,carol) → ancestor(alice,carol) - already present
    parent(alice,bob)  ancestor(bob,dave) → ancestor(alice,dave) - already present
    No new answers added.

Quiescence reached. ancestor cell = {(alice,bob), (bob,carol), (bob,dave),
                                     (alice,carol), (alice,dave)}
```

11.6 A.6 Step 5: Answer Extraction

A query `(solve [ancestor "alice" ?who])` reads the ancestor table cell, filters for entries where `?x = "alice"`, and projects the `?y` values:

```
Results: ?who  {"bob", "carol", "dave"}
```

—

12 Appendix B: End-to-End Query Walkthrough (All Three Layers)

This appendix shows a single query that exercises all three layers of the architecture: PropNetwork (Layer 1), ATMS (Layer 2), and Stratification (Layer 3).

12.1 B.1 Source Program

```
defr edge [?from ?to]
  &> (= ?from "a") (= ?to "b")
  &> (= ?from "b") (= ?to "c")
  &> (= ?from "a") (= ?to "c")
```

```
defr reachable [?x ?y]
  &> (edge ?x ?y)
  &> (edge ?x ?z) (reachable ?z ?y)

;; Negation triggers stratification (Layer 3)
defr unreachable [?x ?y]
  &> (node ?x) (node ?y) (not (reachable ?x ?y))

;; Query with nondeterminism (triggers ATMS, Layer 2)
;; "Find a node that is unreachable from 'a'"
let result := (solve [unreachable "a" ?target])
```

12.2 B.2 Compile-Time: Stratification Check

The predicate dependency graph:

```
edge         (no deps)         → Stratum 0
reachable    → edge (+)        → Stratum 0 (same SCC, all positive)
node         (no deps)         → Stratum 0
unreachable  → reachable ()   → Stratum 1 (negative edge crosses strata)
```

The stratification checker (§7.6) verifies: the negative edge `unreachable --not--> reachable` crosses from Stratum 1 to Stratum 0 — safe.

12.3 B.3 Runtime: Layer 1 — PropNetwork (Stratum 0)

Evaluate `edge`, `reachable`, and `node` to fixed point:

```
1. edge table cell:
   {(a,b), (b,c), (a,c)}

2. reachable table cell (iterate to fixpoint):
   Iteration 1: {(a,b), (b,c), (a,c)} - from edge base case
   Iteration 2: {(a,b), (b,c), (a,c), (a,c)} - (a,b)+(b,c)→(a,c), already present
   Fixpoint: {(a,b), (b,c), (a,c)}

3. node table cell (derived from edge endpoints):
   {a, b, c}
```

All purely monotonic. No ATMS overhead. Layer 1 only.

12.4 B.4 Runtime: Layer 3 — Stratum Boundary

Stratum 0 is complete. Now the runtime evaluates Stratum 1.

`unreachable` uses `not (reachable ?x ?y)`. This reads the *frozen* reachable table from Stratum 0:

```
For each pair (x, y) in node × node:
  (a, a): reachable(a, a) = not found → unreachable(a, a)
  (a, b): reachable(a, b) = found → skip
  (a, c): reachable(a, c) = found → skip
  (b, a): reachable(b, a) = not found → unreachable(b, a)
  (b, b): reachable(b, b) = not found → unreachable(b, b)
  (b, c): reachable(b, c) = found → skip
  (c, a): reachable(c, a) = not found → unreachable(c, a)
  (c, b): reachable(c, b) = not found → unreachable(c, b)
  (c, c): reachable(c, c) = not found → unreachable(c, c)
```

12.5 B.5 Runtime: Layer 2 — ATMS (if needed)

In this example, the query is deterministic (no `amb`). Under the two-tier mode (§5.5), the solver stays in Tier 1 (PropNetwork only) — no ATMS overhead.

If we modified the program to include a choice point:

```
;; "Pick a starting node and find what's unreachable from it"
defr mystery-query [?start ?target]
  &> (node ?start) (unreachable ?start ?target)
```

Now `node ?start` produces 3 alternatives → `amb` activates, upgrading to Tier 2 (full ATMS):

```
Hypothesis h1: ?start = "a"
  → unreachable("a", ?target) under h1
  → Answers: {(a,a)} under h1

Hypothesis h2: ?start = "b"
  → unreachable("b", ?target) under h2
  → Answers: {(b,a), (b,b)} under h2

Hypothesis h3: ?start = "c"
  → unreachable("c", ?target) under h3
```

```
  → Answers: {(c,a), (c,b), (c,c)} under h3

All hypotheses are consistent (no nogoods).
Final answers: union of all worldviews.
```

12.6 B.6 Summary: Which Layer Handles What

Query Aspect	Layer Used	Phase
`edge`, `reachable` tables	1: PropNetwork	Ph 2,6
`not (reachable ...)`	3: Stratification	Ph 7
Stratum evaluation order	3: Stratification	Ph 7
`node ?start` choice points	2: ATMS	Ph 5
Backtracking on contradictions	2: ATMS	Ph 5
Table answer accumulation	1: PropNetwork	Ph 2,6

—

13 Performance Expectations

13.1 Cost Model

The persistent architecture adds a constant factor vs. mutable implementations. Here are the expected costs for each operation:

Operation	Cost (Persistent)	Cost (Mutable)	Ratio
Cell read	O(log n) O(C), C7	O(1) hash lookup	~3x
Cell write (no change)	O(log n) comparison	O(1)	~3x
Cell write (changed)	O(log n) + alloc path	O(1)	~5x
Add propagator	O(log n) × inputs	O(1) × inputs	~3-5x
Backtrack	O(1) — keep old ref	O(n) — deep copy	100x+ better
Snapshot	O(1) — identity	O(n) — deep copy	100x+ better
run-to-quiescence	O(k × log n) per step	O(k) per step	~3-5x

Where:

- n = number of cells in the network (typically 100-10,000)
- k = number of propagator firings to quiescence
- log(10,000) 2.7 — so the "constant" is truly small

13.2 Benchmark Targets (to validate during implementation)

Benchmark	Target	Validates
Create 1000-cell network	< 50ms	CHAMP scaling
Write 1000 values, run to quiescence	< 200ms	Propagation
Backtrack 100 times over 1000-cell net	< 1ms total	Persistence
100-fact ancestor query (tabled)	< 100ms	Tabling
1000-fact Datalog transitive closure	< 500ms	Scalability
`solve` with 100 alternatives (ATMS)	< 1s	ATMS overhead
Same 100 alternatives, Tier 1 only	< 200ms	Two-tier skip

These targets are order-of-magnitude estimates. Actual benchmarks will be added to `tools/benchmark-tests.rkt` as each phase is implemented. The key comparison is: persistent overhead vs. backtracking savings. For any program with search (>1 choice point), the persistent approach wins because each backtrack is O(1) instead of O(n) deep copy.

13.3 When Performance Matters Most

For the logic engine's intended use cases:

- **Type checking**: The elaborator's current metavar system is fast for deterministic type inference. The PropNetwork adds overhead here. The elaborator refactoring (post-Phase 7) should only proceed after benchmarking confirms acceptable overhead.
- **Logic queries**: Search-heavy programs (Prolog-style) benefit enormously from free backtracking. The persistent overhead is amortized over the many backtracks avoided.
- **Datalog**: Bottom-up evaluation with tabling is inherently monotonic — no backtracking needed. Performance depends on propagation efficiency and set operations.

—

14 What This Design Does NOT Cover

14.1 Elaborator Refactoring (Phase 1 of Research Doc)

Refactoring `current-meta-store` to use a persistent PropNetwork internally is recommended by the research document but deferred. The persistent

network architecture makes this refactoring simpler: replace the mutable meta store with a PropNetwork value threaded through elaboration. But it is still a large internal change that can proceed independently after the logic engine exists.

14.2 Galois Connections / Domain Embeddings (Phase 6 of Research Doc)

Modular constraint domains connected via Galois connections are an advanced feature deferred until the basic engine is operational.

14.3 Full Stratified Evaluation Runtime (Phase 4 of Research Doc)

Phase 7 includes a compile-time stratification *check* (§7.6): SCC decomposition, negative edge classification, and rejection of unstratifiable programs. The runtime evaluation of strata (evaluating lower strata to fixpoint before proceeding to higher strata) is also included as part of the solver's query evaluation loop. Full support for aggregation operators (`count`, `min`, `max`, `sum`) between strata is deferred to a follow-up phase.

14.4 CRDTs / Distributed Logic

CRDT-backed collections for distributed actors are long-term goals not addressed here.

14.5 QuickCheck / Property Testing

Executing `:holds` clauses and `:examples` entries requires the logic engine (for proof search). Once the engine exists, this becomes a natural Phase 2 of the Extended Spec Design.

—

15 Architectural Decision: Persistent Networks

15.1 The Problem with Mutable Propagator Networks

The original design proposed mutable Racket structs for PropCell, PropNetwork, TMSCell, and ATMS. This creates the same save/restore problem that plagues the current metavar store:

- **Snapshotting** requires deep copies — O(n) for n cells
- **Backtracking** requires explicit undo — O(mutations) to restore
- **Network mobility** requires serialization — custom ser/deser code
- **Debugging** is harder — state is invisible without explicit inspection

15.2 The Persistent Solution

Making the entire network a persistent value backed by CHAMP maps:

- **Snapshotting** = free (keep a reference — structural sharing)
- **Backtracking** = O(1) (use old reference)
- **Network mobility** = serialize the value (all data is in the struct)
- **Debugging** = the network IS the state (print it, compare it)
- **Cost**: O(log n) per cell write instead of O(1)

Since log(n) 7 for any practical n (up to ~34 billion cells), the cost is bounded by a small constant. For the typical logic engine workload (hundreds to thousands of cells), this is 2-3 levels of CHAMP trie traversal — effectively O(1).

15.3 LVar Elimination

A further simplification: LVars are subsumed by PropNetwork cells. An LVar is just a cell-id in a persistent prop-network, since `net-cell-write` already does join-on-write (inflationary) via the per-cell `merge-fn`. This eliminates ~10 dedicated LVar AST nodes and the separate `lvar.rkt` module.

Before	After
61 AST	51 AST
32 files	27+ files
Separate LVar, LVar-Set, LVar-Map	Cells with SetLattice, MapLattice
Mutable cells + scheduler	Persistent network, pure loop
`save/restore` for backtracking	Keep old reference

—

16 Key Lessons from Prior Work

(Gathered from CLAUDE.md MEMORY and implementation experience)

1. **12+ file pipeline for new AST nodes** — every new `expr-*` touches syntax.rkt, typing-core.rkt, reduction.rkt, elaborator.rkt, zonk.rkt, pretty-print.rkt, qtt.rkt, substitution.rkt, unify.rkt, + tests.
2. **Per-command parameter isolation** — all runtime state (meta stores, warning accumulators, table stores) must be parameterized and reset per-command in `driver.rkt`.
3. **Trait dictionary dispatch** — single-method traits: dict IS the function. Multi-method: nested Sigma. Dict names: `TypeArg--TraitName--dict`.
4. `collect-constraint-values` — the right collector for list-valued metadata like `:examples`, `:laws`, `:tabled`.
5. **Generic `rewrite-implicit-map` branch** — WS-mode keyword blocks need to call `rewrite-implicit-map` at dispatch. The property/functor bugs taught this lesson.
6. **Transient pattern** — generic wrappers (`expr-transient`, `expr-persist`)
   - type-specific variants. Follow this for cell/lvar generics.
7. **Test file size** — keep under ~20 cases / ~30s for parallel execution. Split large test files into -01, -02 parts.
8. **Update dep-graph.rkt** — every new .rkt file needs an entry in `tools/dep-graph.rkt` for incremental testing to work.

—

17 References

Organized by phase relevance:

17.1 Phase 1 (Lattice)

- Cousot & Cousot, "Abstract Interpretation: A Unified Lattice Model" (POPL, 1977)
- Tarski, "A Lattice-Theoretical Fixpoint Theorem" (Pacific J. Math., 1955)

17.2 Phase 2 (Propagators)

- Radul & Sussman, "The Art of the Propagator" (MIT TR, 2009)
- Radul, *Propagation Networks* (PhD thesis, MIT, 2009)
- Hellerstein, "Keeping CALM" (CACM, 2020)

17.3 Phase 3 (LVars)

- Kuper & Newton, "LVars: Lattice-Based Data Structures for Deterministic Parallelism" (FHPC, 2013)
- Kuper et al., "Freeze After Writing" (POPL, 2014)

17.4 Phase 4 (UnionFind)

- Conchon & Filliâtre, "A Persistent Union-Find Data Structure" (ML Workshop, 2007)

17.5 Phase 5 (ATMS)

- de Kleer, "An Assumption-Based TMS" (AI Journal, 1986)

17.6 Phase 6 (Tabling)

- Swift & Warren, "XSB: Extending Prolog with Tabled Logic Programming" (TPLP, 2012)
- Chen & Warren, "Tabled Evaluation with Delaying" (JACM, 1996)
- Madsen et al., "From Datalog to Flix" (PLDI, 2016)

17.7 Phase 7 (Surface)

- Arntzenius & Krishnaswami, "Datafun: A Functional Datalog" (ICFP, 2016)
- Fruhwirth, *Constraint Handling Rules* (CUP, 2009)